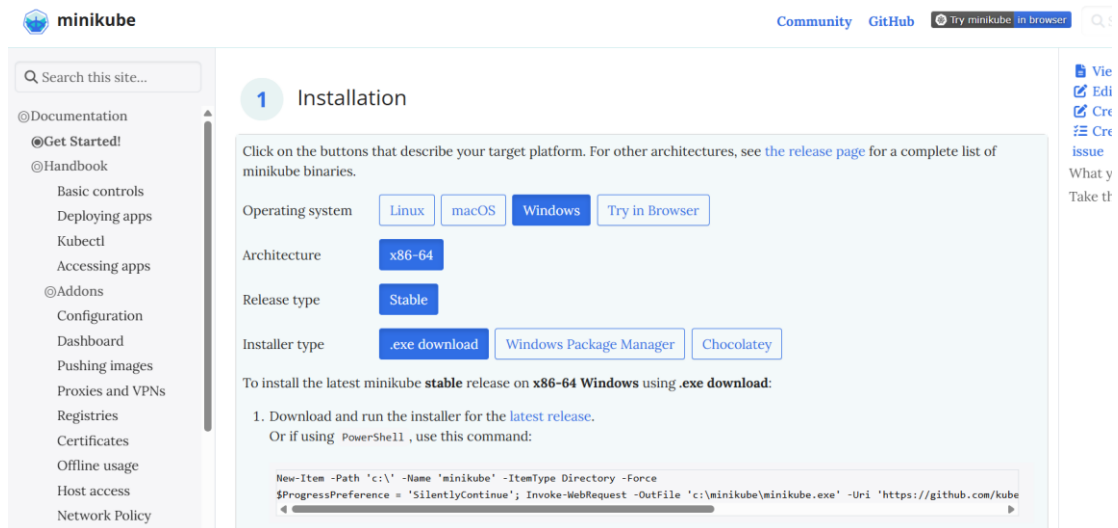


K8s-01

1. Setup Minikube in your local machine.

- Open Google and search Minikube install and select which Operating System and copy command.



The screenshot shows the Minikube website's installation page. The 'Operating system' is set to 'Windows', 'Architecture' to 'x86-64', and 'Release type' to 'Stable'. The 'Installer type' is set to '.exe download'. The instructions state: 'To install the latest minikube stable release on x86-64 Windows using .exe download: 1. Download and run the installer for the latest release. Or if using PowerShell, use this command:'. A PowerShell command is provided in a code block: `New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force; $ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri 'https://github.com/kube`

- Open PowerShell and paste the command

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO> New-Item -Path 'c:\' -Name 'minikube' -ItemType Directory -Force

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----          23-06-2026   16:43           minikube

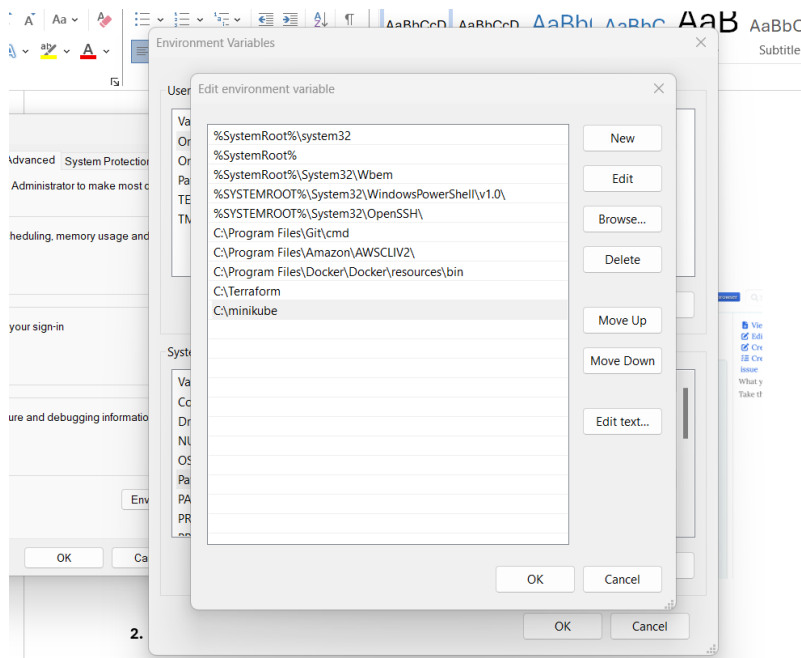
PS C:\Users\LENOVO> $ProgressPreference = 'SilentlyContinue'; Invoke-WebRequest -OutFile 'c:\minikube\minikube.exe' -Uri 'https://github.com/kubernetes/minikube/releases/latest/download/minikube-windows-amd64.exe' -UseBasicParsing
```

- Now do Environmental variables and check minikube version

```
PS C:\Users\LENOVO> $oldPath = [Environment]::GetEnvironmentVariable('Path', [EnvironmentVariableTarget]::Machine)
PS C:\Users\LENOVO> if ($oldPath.Split(';') -notcontains 'C:\minikube'){
>> [Environment]::SetEnvironmentVariable('Path', $('{0};C:\minikube' -f $oldPath), [EnvironmentVariableTarget]::Machine)
>> }
Exception calling "SetEnvironmentVariable" with "3" argument(s): "Requested registry access is not allowed."
At line:2 char:51
+ ... etEnvironmentVariable('Path', $('{0};C:\minikube' -f $oldPath), [Envi ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : SecurityException

PS C:\Users\LENOVO> C:\minikube\minikube.exe version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
PS C:\Users\LENOVO> $env:Path += ";C:\minikube"
PS C:\Users\LENOVO> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

- Open Environmental variables and in variables select path and select edit and copy minikube location path and paste it



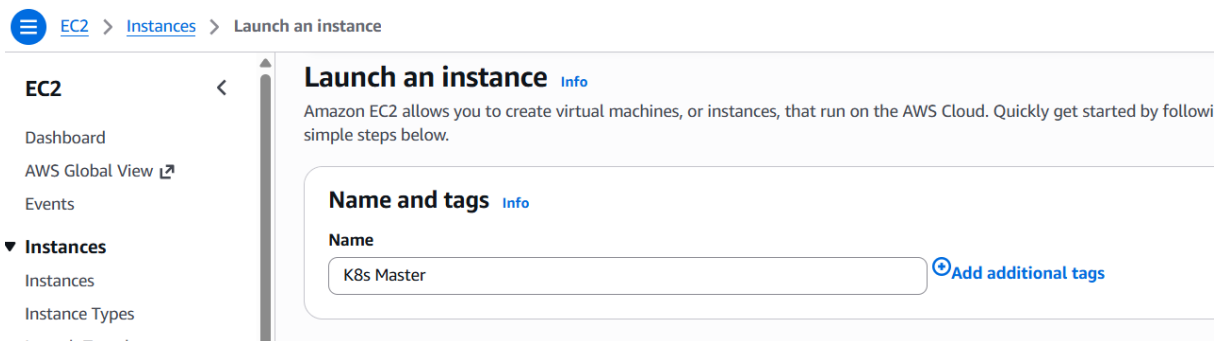
- Open terminal use command minikube start and it starts download and check nodes

```
PS C:\Users\LENOVO> minikube start
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 22H2
* Kubernetes 1.35.1 is now available. If you would like to upgrade, specify: --kubernetes-version=v1.35.1
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Updating the running docker "minikube" container ...
* Preparing Kubernetes v1.31.0 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass

! C:\Program Files\Docker\Docker\resources\bin\kubectl.exe is version 1.34.1, which may have incompatibilities with Kubernetes 1.31.0.
  - Want kubectl v1.31.0? Try 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\Users\LENOVO> kubectl get nodes
NAME          STATUS    ROLES          AGE      VERSION
minikube      Ready     control-plane   7h42m    v1.31.0
PS C:\Users\LENOVO>
```

2. Setup k8s master and two worker nodes on ubuntu.

- Launch new instance with name K8s Master



- Select image is ubuntu

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Ubuntu Server 26.04 LTS (HVM), SSD Volume Type
ami-0b6d9d3d33ba97d99 (64-bit (x86)) / ami-0bc7f2dbdcc6b5303 (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

- Instance type c7i-flex. large

▼ Instance type [Info](#) | [Get advice](#)

Instance type

c7i-flex.large

Family: c7i-flex 2 vCPU 4 GiB Memory Current generation: true
On-Demand Windows base pricing: 0.17219 USD per Hour
On-Demand RHEL base pricing: 0.11359 USD per Hour
On-Demand Ubuntu Pro base pricing: 0.08829 USD per Hour
On-Demand Linux base pricing: 0.08479 USD per Hour
On-Demand SUSE base pricing: 0.14109 USD per Hour

Free tier eligible

[All generations](#)

[Compare instance types](#)

[Additional costs apply for AMIs with pre-installed software](#)

- Key pair new

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

[Create new key pair](#)

- networking settings

▼ Network settings [Info](#)

[Edit](#)

Network

vpc-07106299541594725

Subnet

No preference (Default subnet in any availability zone)

Auto-assign public IP

Enable

Firewall (security groups)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group

☒ Select existing security group

Common security groups

Select security groups

default sg-0075c32a4bd04f8cd
VPC: vpc-07106299541594725

[Compare security group rules](#)

- same create other 2 work nodes
- 3 instances created

Instances (3) Info Last updated less than a minute ago Connect Instance state Actions Launch instances

Saved filter sets: Running

Instance state = running Clear filters

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	K8s Master	i-0cfb60b76f1d1f8a3	Running	c7i-flex.large	3/3 checks passed	us-east-1d	ec2-34-235-113-47.co...
☐	K8s Worknode1	i-01269228ce8c0134a	Running	c7i-flex.large	3/3 checks passed	us-east-1d	ec2-13-217-41-184.co...
☐	K8s Worknode2	i-08bdc0ed5669e3a5f	Running	c7i-flex.large	3/3 checks passed	us-east-1d	ec2-3-85-186-107.com..

- Change hostnames for master and worknodes

```
ubuntu@Master:~$ sudo su -
root@Master:~# cat /etc/hostname
Master
root@Master:~#
```

- Now install docker in master and same with worknodes also

```
root@Master:~# sudo sed -i 's / swap / s/^/#/' /etc/fstab
root@Master:~# sudo apt update -y
sudo apt install -y docker.io
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu InRelease [137 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu InRelease [136 kB]
Get:4 http://security.ubuntu.com/ubuntu InRelease [137 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu InRelease [16.3 MB]
Get:6 http://security.ubuntu.com/ubuntu InRelease [228 kB]
Get:7 http://security.ubuntu.com/ubuntu InRelease [1.0 kB]
Get:8 http://security.ubuntu.com/ubuntu InRelease [amd64 Components]
```

- Now do modprobe in master

```
sudo modprobe br_netfilter
root@Master:~# cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
EOF
net.bridge.bridge-nf-call-iptables=1
net.bridge.bridge-nf-call-ip6tables=1
net.ipv4.ip_forward=1
root@Master:~# sudo sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /usr/lib/sysctl.d/10-coredump-debian.conf ...
```

- Now create mkdir contained

```
root@Master:~# sudo mkdir -p /etc/containerd
root@Master:~# containerd config default | sudo tee /etc/containerd/config.toml
version = 3
root = '/var/lib/containerd'
state = '/run/containerd'
temp = ''
disabled_plugins = []
required_plugins = []
oom_score = 0
imports = ['/etc/containerd/conf.d/*.toml']

[grpc]
address = '/run/containerd/containerd.sock'
```

- Check versions of kubeadm, kubectl, kubelet

```
root@Master:~# kubectl version
kubectl version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"31", GitVersion:"v1.31.14", Git
Commit:"5e00b99bac504844579ec74886b6cc5c9611ca19", GitTreeState:"clean", BuildDa
te:"2025-11-11T20:23:36Z", GoVersion:"go1.24.9", Compiler:"gc", Platform:"linux/
amd64"}
Client Version: v1.31.14
Kustomize Version: v5.4.2
Kubernetes v1.31.14
```

- Pod Network cidr only in master

```
root@Master:~# sudo kubeadm init --pod-network-cidr=192.168.0.0/16
I0624 10:27:59.867805 5962 version.go:261] remote version is much newer: v1.3
6.2; falling back to: stable-1.31
[init] Using Kubernetes version: v1.31.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your inte
rnet connection
[preflight] You can also perform this action beforehand using 'kubeadm config im
ages pull'
```

```
root@Master:~# kubectl logs -n kube-system calico-node-4vhkm
Defaulted container "calico-node" out of: calico-node, upgrade-ipam (init), inst
all-cni (init), ebpf-bootstrap (init)
Error from server (BadRequest): container "calico-node" in pod "calico-node-4vhk
m" is waiting to start: PodInitializing
root@Master:~# sudo systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; prese
   Active: active (running) since Wed 2026-06-24 10:18:38 UTC; 32min ago
   Invocation: db2d9f3c1904ef38db72a79b65d45b4
     Docs: https://containerd.io
    Main PID: 4764 (containerd)
      Tasks: 83
     Memory: 1021.3M (peak: 1G)
        CPU: 14.716s
    CGroup: /system.slice/containerd.service
```

```
root@Master:~# export KUBECONFIG=/etc/kubernetes/admin.conf
root@Master:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
master        NotReady control-plane 76s   v1.31.14
root@Master:~# kubectl apply -f https://raw.githubusercontent.com/projectcalico/
calico/master/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
serviceaccount/calico-node created
serviceaccount/calico-cni-plugin created
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalic
o.org created
customresourcedefinition.apiextensions.k8s.io/bgpfilters.crd.projectcalico.org c
```

```
root@Master:~# kubectl get pods -n kube-system -w
NAME                                READY   STATUS    RESTARTS   AGE
calico-kube-controllers-57d9bcf647-2984x  1/1     Running   0           33s
calico-node-tjd94                      1/1     Running   0           33s
coredns-7c65d6cfc9-f44rv              1/1     Running   0           28m
coredns-7c65d6cfc9-v4ggt              1/1     Running   0           28m
etcd-master                           1/1     Running   0           28m
kube-apiserver-master                  1/1     Running   0           28m
kube-controller-manager-master         1/1     Running   0           28m
kube-proxy-sf4dx                       1/1     Running   0           28m
kube-scheduler-master                  1/1     Running   0           28m

kubectl get nodes
ACroot@Master:~#
root@Master:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
master        Ready     control-plane 29m   v1.31.14
```

- Now connect with worknodes to master

```
root@worknode1:~# which conntrack
/usr/sbin/conntrack
root@worknode1:~# sudo kubeadm join 172.31.37.70:6443 \
--token ydg99x.hk7alg6n5j7ej9ac \
--discovery-token-ca-cert-hash sha256:4de0951d6f354cf75e4a90824c2fb6a0907e8722714999b2a2335858331eb47e
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system g
et cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.y
aml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/ku
belet/kubeadm-flags.env"
```

```
root@worknode2:~# sudo kubeadm join 172.31.37.70:6443 \
--token ydg99x.hk7alg6n5j7ej9ac \
--discovery-token-ca-cert-hash sha256:4de0951d6f354cf75e4a90824c2fb6a0907e8722714999b2a2335858331eb47e
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system g
et cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.y
aml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/ku
belet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz.
This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.808306ms
[kubelet-start] Waiting for the kubelet to perform the TLS bootstrap
```

- After connecting work nodes to master check in master ready or not

```
root@master:~# kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE     VERSION
master        Ready     control-plane  52m     v1.31.14
worknode1     Ready     <none>      8m57s   v1.31.14
worknode2     Ready     <none>      22s     v1.31.14
root@master:~# kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE     VERSION    INTERNAL-IP    EXTERNAL-IP    OS-I
MAGE        KERNEL-VERSION    CONTAINER-RUNTIME
master        Ready     control-plane  52m     v1.31.14    172.31.37.70    <none>         Ubun
tu 26.04 LTS  7.0.0-1006-aws    containerd://2.2.2
worknode1     Ready     <none>      9m35s   v1.31.14    172.31.40.125   <none>         Ubun
tu 26.04 LTS  7.0.0-1006-aws    containerd://2.2.2
worknode2     Ready     <none>      60s     v1.31.14    172.31.44.32    <none>         Ubun
tu 26.04 LTS  7.0.0-1006-aws    containerd://2.2.2
root@master:~# kubectl get pods -A
NAMESPACE     NAME                                     READY   STATUS    RESTARTS   AGE
kube-system   calico-kube-controllers-57d9bcf647-2984x  1/1     Running   0           24m
kube-system   calico-node-4875q                         1/1     Running   0           9m43s
kube-system   calico-node-tjd94                         1/1     Running   0           24m
kube-system   calico-node-www6r                         1/1     Running   0           68s
kube-system   coredns-7c65d6cfc9-f44rv                 1/1     Running   0           52m
kube-system   coredns-7c65d6cfc9-v4gtt                 1/1     Running   0           52m
```

3. Run one nginx pod.

- Run kubectl nginx image

```
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
PS C:\Users\LENOVO> kubectl run nginx --image=nginx
Error from server (AlreadyExists): pods "nginx" already exists
PS C:\Users\LENOVO> kubectl get pods
NAME     READY   STATUS    RESTARTS   AGE
nginx    1/1     Running   1 (9m10s ago)  7h43m
PS C:\Users\LENOVO> kubectl expose pod nginx --type=NodePort --port=80
Error from server (AlreadyExists): services "nginx" already exists
PS C:\Users\LENOVO> kubectl get svc
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes   ClusterIP   10.96.0.1     <none>         443/TCP          7h48m
nginx        NodePort    10.98.180.18  <none>         80:32131/TCP     7h43m
```

- Check service nginx on which port

```

PS C:\Users\LENOVO> minikube service nginx

```

NAMESPACE	NAME	TARGET PORT	URL
default	nginx	80	http://192.168.49.2:32131

```

* Starting tunnel for service nginx.

```

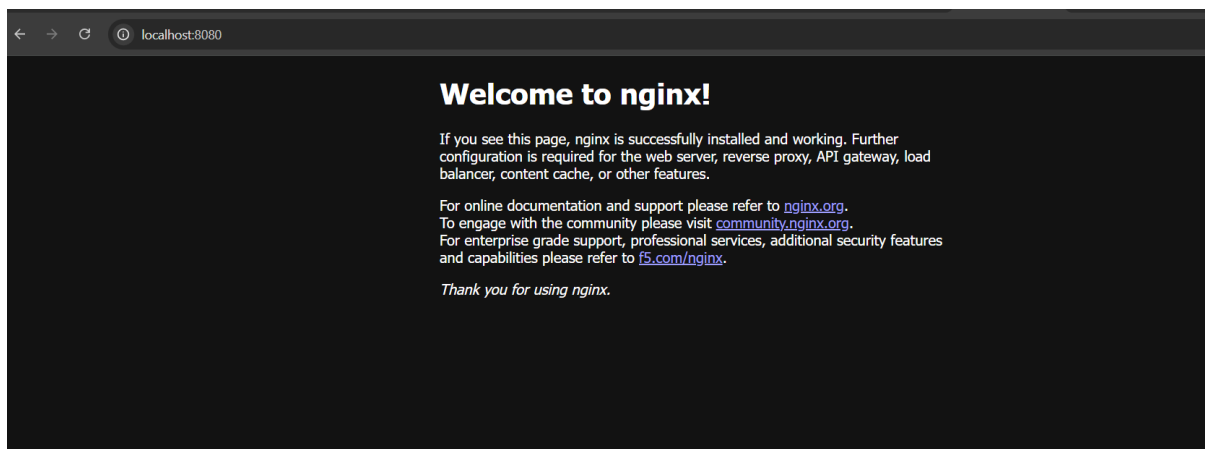
NAMESPACE	NAME	TARGET PORT	URL
default	nginx		http://127.0.0.1:60085

```

* Opening service default/nginx in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

```

- Check in browser nginx page is opening or not



4. Mugup Master and slave components on k8s.

Master Components:

1. kube-apiserver

- Entry point of Kubernetes.
- Receives all requests from kubectl.
- Communicates with etcd and other components.

Example:

- **kubectl get pods (Request goes to kube-apiserver)**

2. etcd

- Key-value database.
- Stores cluster state and configuration.
- Source of truth for Kubernetes.

Example:

- **Pod information**
- **Node information**

- **Secrets**
- **ConfigMaps**

3. kube-scheduler

- Decides on which node a pod should run.
- Checks CPU, Memory, Labels, Taints, etc.

Example:

- **Nginx pod created**
- **Scheduler places it on a worker node**

4. kube-controller-manager

- Runs controllers.
- Maintains desired state.

Examples:

- **Node Controller**
- **Replication Controller**
- **Endpoint Controller**

Worker Node (Slave Node) Components

1. kubelet

- Agent running on every worker node.
- Communicates with API Server.
- Creates and manages pods.

Example:

- **API Server says "Run nginx pod"**
- **kubelet starts the pod.**

2. kube-proxy

- Handles networking.
- Maintains Service communication.
- Implements ClusterIP, NodePort, Load Balancer networking.

Example:

- **Service routes traffic to nginx pod.**

3. Container Runtime

- Runs containers.

Examples:

- **containerd**
- **Docker**
- **CRI-O**

In your Minikube setup, containers are running through the container runtime.

4. Pods

- Smallest deployable unit in Kubernetes.
- Contains one or more containers.

Example:

- **kubectl get pods**